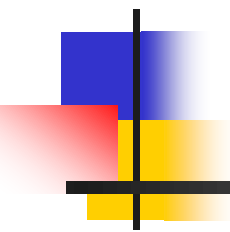


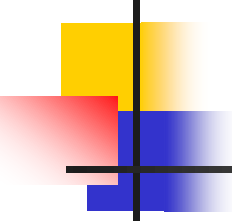
6. Interfaces and Packages



Interfaces

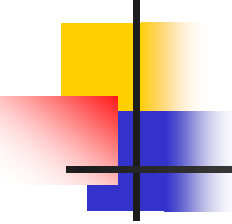
Package - access control

Creating our own package



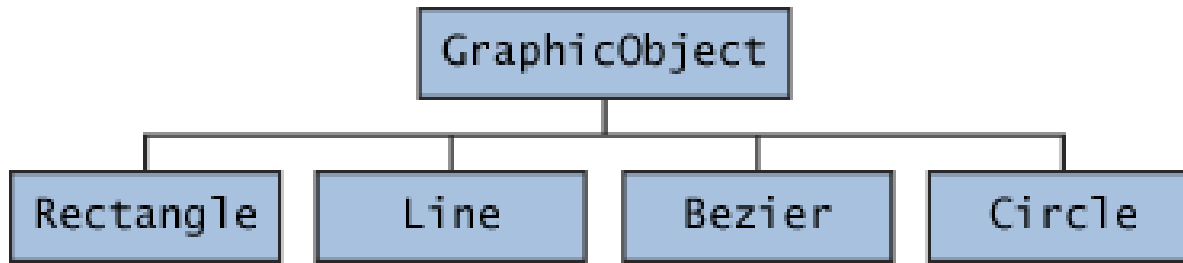
Summary: Inheritance

- Except for the Object class, a class has exactly one direct superclass. A class inherits fields and methods from all its super classes, whether direct or indirect.
- An abstract class can only be subclassed; it cannot be instantiated. An abstract class can contain abstract methods—methods that are declared but not implemented. Subclasses then provide the implementations for the abstract methods.



Summary: Inheritance

- You can prevent a class from being subclassed by using the final keyword in the class's declaration. Similarly, you can prevent a method from being overridden by subclasses by declaring it as a final method.

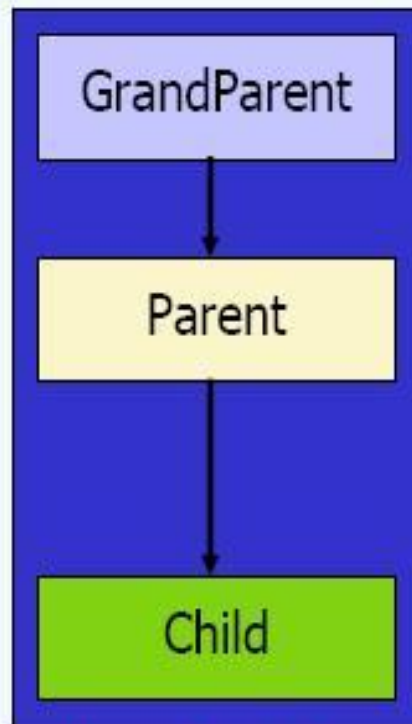


```
abstract class GraphicObject {
    int x, y;
    ...
    void moveTo(int newX, int
        newY) {
        ...
    }
    abstract void draw();
    abstract void resize();
}
```

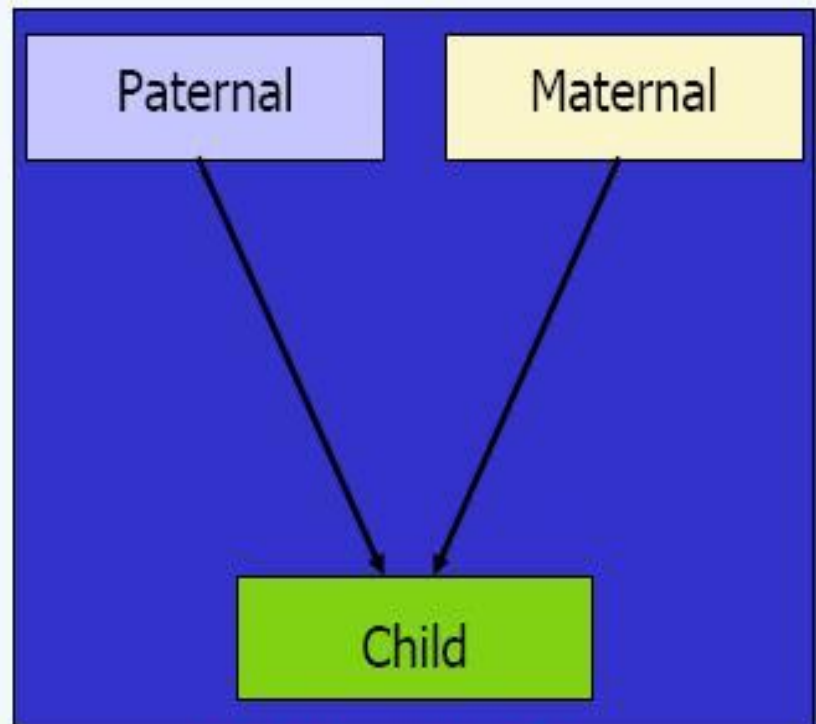
```
class Circle extends GraphicObject {
    void draw() {
        ...
    }
    void resize() {
        ...
    }
}
```

```
class Rectangle extends GraphicObject {
    void draw() {
        ...
    }
    void resize() {
        ...
    }
}
```

Interfaces in Java

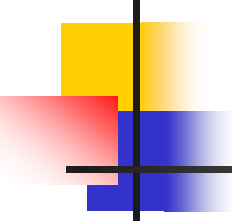


Allowed in Java



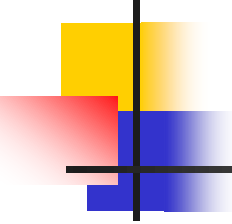
Not Allowed in Java





Interfaces in Java

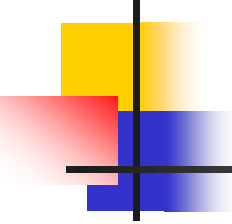
- An ***interface*** is a set of predefined methods to be implemented by one or more classes in future.
- An Interface will just give what the method should do, but it will not give the implementation for it.
- This helps the programmer to write his own logic in his class, which implements the interface.
- The methods in an interface will have no body and it just mention the method signature.



Interface

- A completely abstract class
- Only constants and abstract methods are allowed
- The ‘*class*’ keyword is replaced by ‘*interface*’

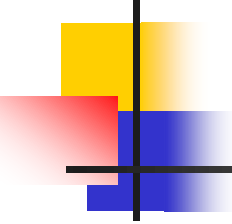
```
public interface Drawable {  
    void draw();  
    double getArea();  
}
```



Implementing an Interface

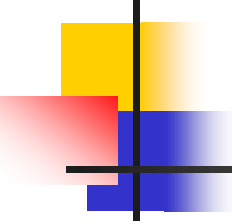
- The general form of implementing an interface is:

```
class classname [extends superclass] implements  
    interface1 [, interface2, . . .] {  
    // body.....  
}
```



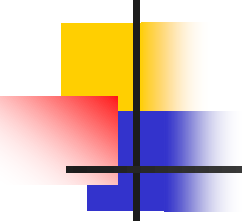
Interface - cont'd

- All members are implicitly public; no need to supply access modifiers (and supplying any other than public is an error).
- Cannot be instantiated.
- Other classes may *'implement'* an interface.



Example:

```
public class Circle extends Shape implements Drawable {  
    public void draw() { ... }  
  
    public double getArea() {  
        return radius*radius*Math.PI;  
    }  
  
    public display() { . . . }  
}
```



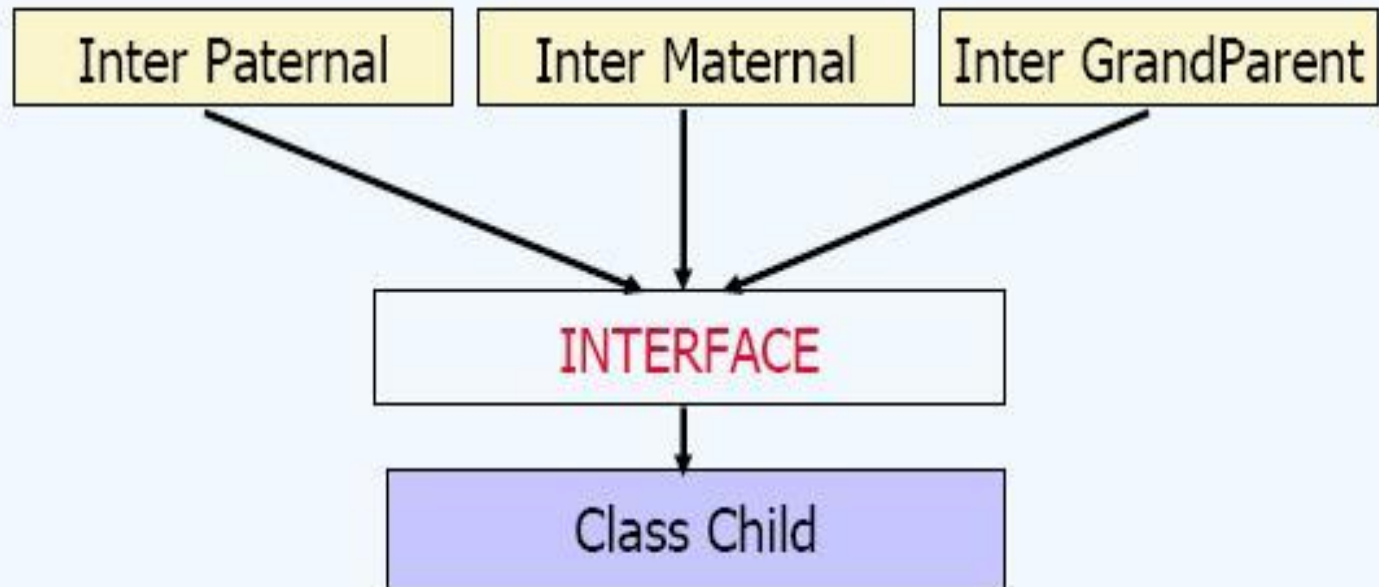
```
abstract class GraphicObject {  
    int x, y;  
    ...  
    void moveTo(int newX, int  
        newY) {  
        ...  
    }  
    abstract void draw();  
    abstract void resize();  
}
```

```
interface GraphicObject {  
  
    void draw();  
    void resize();  
}
```

```
class Circle extends GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```

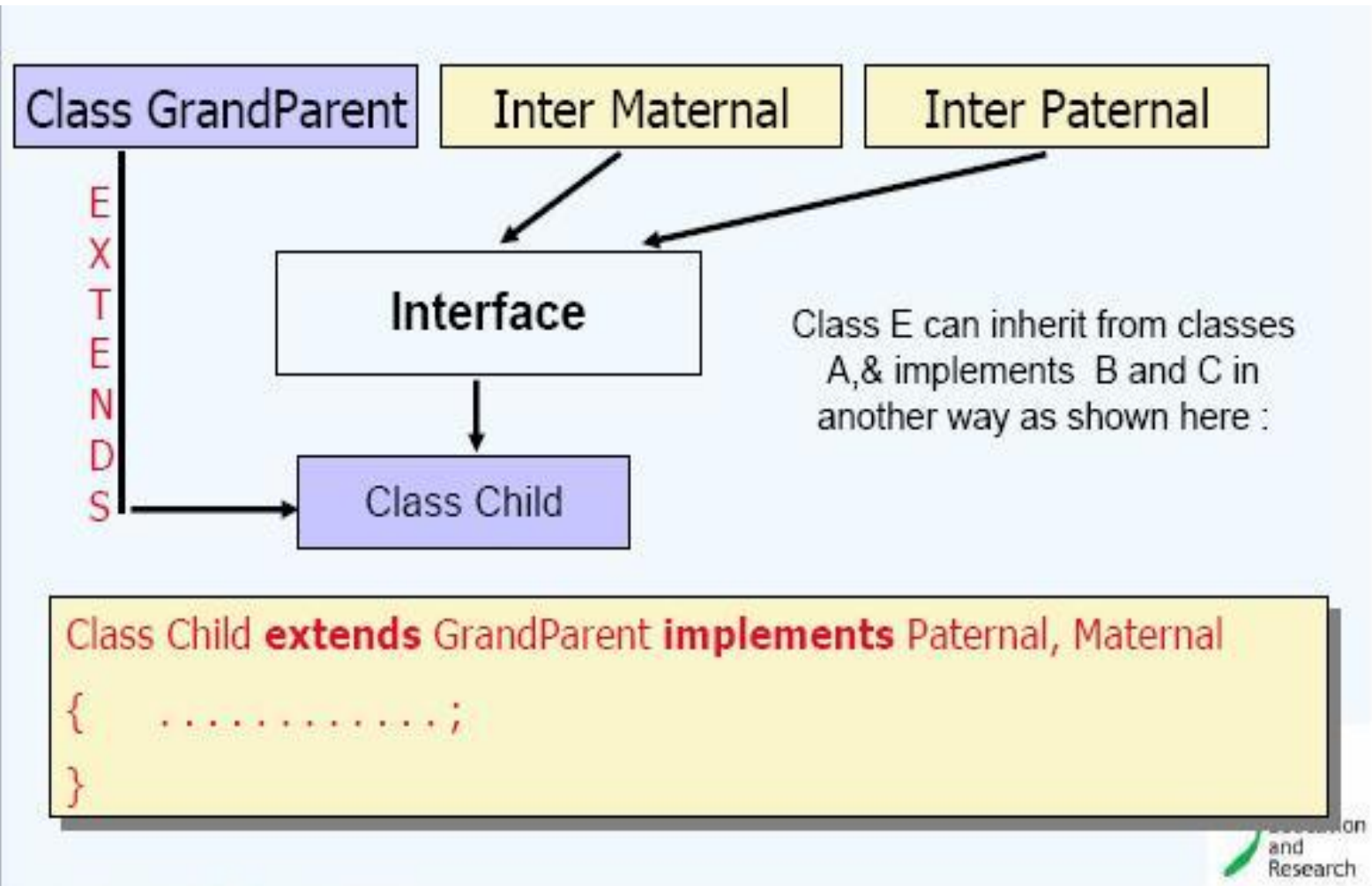
```
class Circle implements GraphicObject {  
    void draw() {  
        ...  
    }  
    void resize() {  
        ...  
    }  
}
```

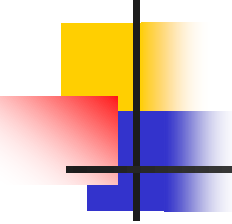
Implementing Multiple Inheritance in Java



```
Class Child implements Paternal, Maternal, GrandParent  
{ .....;  
}
```

Implementing Multiple Inheritance in Java

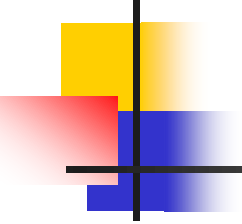




Abstract Class v/s Interfaces

- | | |
|--|---|
| <ul style="list-style-type: none">■ Specifies the full set of methods for an object.■ Implements none, some or all methods.■ Useless without being subclassed. | <ul style="list-style-type: none">■ Specifies a subset of methods for an object.■ Implements none of its methods.■ Useless without being implemented. |
|--|---|

Both Abstract classes and Interfaces cannot be instantiated.



Summary:

Classes, Abstract Classes, and Interfaces

- **Classes**

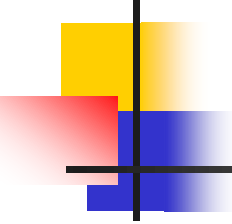
- All declared methods must be defined.
- No restriction on member variables.

- **Abstract Classes**

- Some methods may be defined.
- No restriction on member variables.

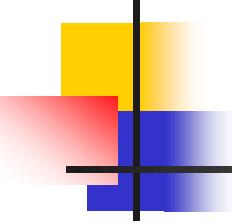
- **Interfaces**

- Methods are only declared.
- Member variables must be static and final.



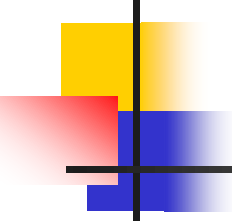
Polymorphism Example

```
public class Animal {  
    public void sound() { }  
}  
  
public class Dog extends Animal {  
    public void sound() {  
        System.out.println("Woof !!");  
    }  
}  
  
public class Duck extends Animal {  
    public void sound() {  
        System.out.println("Quack!!");  
    }  
}
```



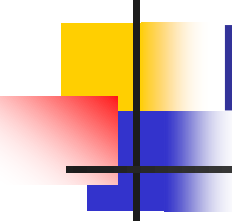
Polymorphism Example cont'd

```
public class TryPolymorphism {  
    public static void main(String args[]) {  
        // theAnimals is an array of superclass references  
        Animal[] theAnimals= { new Dog(), new Duck() };  
  
        Animal petChoice = theAnimals[0];  
        petChoice.sound();                // calls Dog's method  
  
        petChoice = theAnimals[1];  
        petChoice.sound();                // calls Duck's method  
    }  
}
```



Polymorphism via Interfaces

- An object of a class implementing an interface may be treated as an object of type corresponding to the interface.
- Different classes support the same set of operations by implementing the same interface.
- At runtime, any of the implementing classes can be used



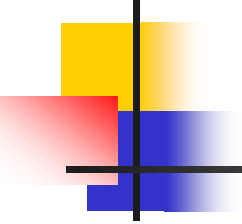
Example:

```
public interface Shape {  
    public double getArea();  
}
```

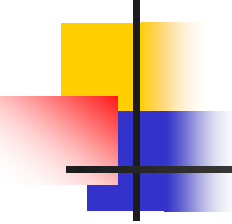
```
public class Circle implements Shape {  
    public double getArea() { return PI * r * r ; }  
}
```

```
public class Rectangle implements Shape {  
    public double getArea() { return height * width ; }  
}
```

```
public class Triangle implements Shape {  
    public double getArea() { return 0.5 * base * height ; }  
}
```

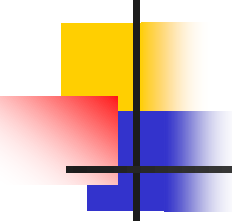


```
public class ShapeAreaFinder {  
    public static void main(String[] args) {  
        Shape [] theShapes = { new Circle(1, 2, 3), new  
                                Rectangle(10, 23), new Triangle(12, 20) };  
  
        System.out.println(theShapes[0].getArea());  
        System.out.println(theShapes[1].getArea());  
        System.out.println(theShapes[2].getArea());  
    }  
}
```



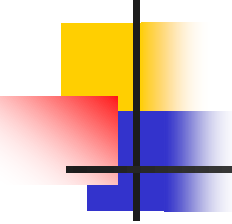
Packages: Why? What?

- Reusability of code is one of the most important requirements in the software industry.
- Reusability saves time, effort, and also ensures consistency.
- A class once developed can be reused by any number of programs wishing to incorporate the class in that particular program.



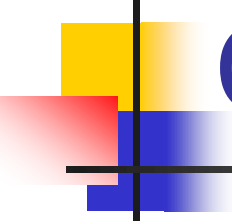
Features of Packages

- Organize our classes into smaller units and make it easy to locate and use.
- Avoid naming conflicts.
- Packages allow us to protect our classes, data and methods in a larger way than on a class-to-class basis.
- Package names can be used to identify our classes.



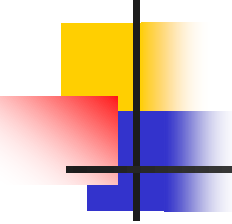
Java Packages

- Classes are organized into packages
- Basically, a package is a set of related classes
- Default accessibility is package-wide:
That is only classes from the same package can see or use data members or methods (or classes!) with no accessibility modifier.



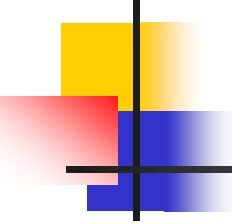
Concept of Packages

- In Java, the code which can be reused by other programs is put into a “Package”.
- A Package is a collection of classes, interfaces and/or other packages.
- Packages are essentially a means of organizing classes together as groups.



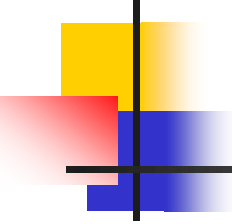
Reasons for package

- Programmers can easily determine that these types are related
- Programmers know where to find types that can provide type related functions
- The names of your types won't conflict with the type names in other packages because the package creates a new namespace



Packages, con't

- The package a class belongs to is defined by a statement at the very top of the file:
package <package name>;
- If no package statement, then it belongs to the default *'nameless'* package.
- To use classes in another package, they must be imported.



Importing Classes

- Tells the compiler we are going to use classes from another package.

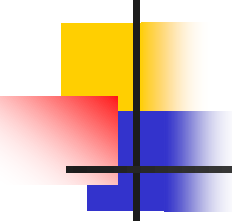
- Syntax:

import <package name>.<class name>;

- To import all classes in a package:

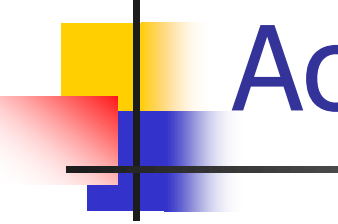
import <package name>.;*

- Comes before class definition, after package statement.



Example Java Packages

- Package *java.lang*
 - Basic classes of the language – System, String, etc
 - Automatically imported
- Package *java.io*
 - Classes for input and output – streams, file system
- Package *java.util*
 - Provides utility classes – complex data structures, etc



Access Specifiers

	Private	The default Specifier ""	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Sub Class	No	Yes	Yes	Yes
Same Package Non-Sub Class	No	Yes	Yes	Yes
Different Package Sub Class	No	No	Yes	Yes
Different Package Non-Sub Class	No	No	No	Yes 